uncluttered syntax, the Markaby example shown earlier barely looks like Ruby at all.

That's why Ruby programmers tend to use Ruby where Java programmers would use an external language or an XML file. It's easier to adapt Ruby to your own needs than it is to adapt Java. As an example, consider build languages. The standard build languages for Java and C (Ant and make, respectively) are external DSLs, while the standard build language for Ruby (Rake) is just a Ruby library—an internal DSL.

DSLs and Metaprogramming

At the beginning of this book, we defined metaprogramming as "writing code that writes code" (or, if you want to be more precise, "writing code that manipulates the language constructs at runtime"). Now that you know about DSLs, you have another definition of metaprogramming: "designing a domain-specific language and then using that DSL to write your program."

This is a book about the first definition, not a book about DSLs. To write a DSL, you have to deal with a number of challenges that are outside the scope of this book. You have to understand your domain, care about your language's user-friendliness, and carefully evaluate the constraints and tradeoffs of your grammar. While writing this book, I opted to keep this particular can of worms shut.

Still, metaprogramming and DSLs have a close relationship in the Ruby world. To build an internal DSL, you must bend the language itself, and doing so requires many of the techniques described in this book. Put another way, metaprogramming provides the bricks that you need to build DSLs. If you're interested in internal Ruby DSLs, this book contains information that's important for you.